

University Enrollment System

A Brief Technical Explainer

Prepared for Salman Adnan's portfolio

*A concise, accurate summary for quick review before a demo or interview. For full depth — every function, every edge case, every diagram — see the companion **deep-dive.pdf** in this same folder. Everything here comes from the project's own source code, README, and proposal document; anything not directly confirmed is marked **[flagged]**.*

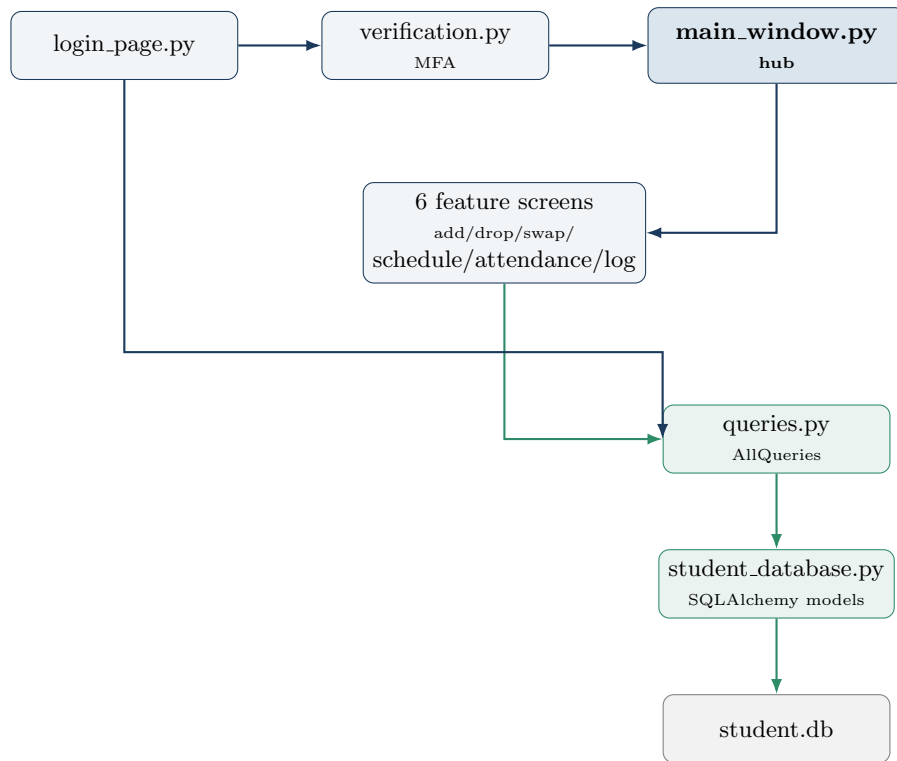
1 What it is

A desktop course-registration application: PyQt6 for the GUI (Python bindings for the Qt6 windowing toolkit), SQLAlchemy as the ORM (object-relational mapper, letting code treat database rows as Python objects) over a single-file SQLite database (**student.db**). A student logs in with a password, confirms a six-digit code sent by email (or printed to the terminal if no mail credentials are configured), then can enroll in, add, drop, or swap course sections with schedule-clash and seat-capacity checks, view a day/week class schedule (exportable to Excel), mark attendance, and review an activity log of everything they have done.

Built as the semester project for the Databases course (CS/CE 355/373) at Habib University, by a two-person team: **Salman Adnan** and **Shayan Wasif**. Per the README, Salman's areas were the email-verification (MFA) login step, the attendance module and its email confirmations, Docker packaging, and cross-screen integration/bug fixing.

The project's own proposal document frames it as a lightweight prototype of three improvements to Habib's real student-records platform (PeopleSoft Campus Solutions, "PSCS" — the source of the **pscs-app** Docker image name): adding multi-factor login, automated attendance notifications, and a course-change activity log. All three are implemented as email-based features; the proposal's mention of SMS as an alternative channel was not built **[flagged]**.

2 Architecture at a glance



Every screen is a `QMainWindow` subclass whose visual layout is loaded at runtime from a matching Qt Designer `.ui` XML file (`uic.loadUi('name.ui', self)`) rather than built in Python code. All database access, from every screen, goes through one class: `AllQueries` in `queries.py`, the project's data-access layer. That class in turn uses SQLAlchemy models defined in `student_database.py` (`Student`, `Course`, `CourseList`, `Filter`, `Activity`) against the `student.db` SQLite file. Two files with no live effect on the app ship in the repo but are never imported: `courses_database.py` (a superseded, schema-incompatible seeding script) and `add_remove_interaction.ui` (an unused layout file) [flagged as dead code, confirmed by grepping for imports/loadUi calls].

3 Data model, in brief

Student (id, name, email, password — stored in plaintext, dob, major) → **Course** (join table: which student is linked to which catalog section, with an `is_temp` flag distinguishing a cart item from a confirmed enrollment) → **CourseList** (the catalog: one row per section, with its time, room, instructor, and a capacity counter that flips a section's `status` to closed at zero seats). **Activity** logs every add/drop/swap/filter-change/export/attendance action with a timestamp. **Filter** and two foreign-key columns on **Course** (`swap_id`, `remove_id`) exist in the schema but, confirmed by reading every query in `queries.py`, are never read or written anywhere [flagged as unused schema].

4 Feature walkthrough

- **Login + MFA:** password is checked with a plain string comparison; a six-digit code is then emailed via Gmail SMTP (or printed to the terminal, with an explanatory dialog, if `ATTENDANCE_SENDER_EMAIL/ATTENDANCE_SENDER_PASSWORD` are not set in the environment). Entering the right code opens the main hub.

- **Add courses:** browse the open catalog, add sections to a provisional "cart" (`is_temp=True`), then confirm on a second screen, which finalizes the cart (`is_temp=False`), decrements each section's capacity, and logs one "Enroll" activity. Both this flow and swap (below) run the same clash-detection check: convert each section's start/end time to seconds-since-midnight, and flag an overlap if the new section's day letters intersect an existing section's and the time ranges start inside, end inside, or fully contain one another.
- **Drop courses:** pick an enrolled section, confirm, delete it, restore one seat of capacity, log a "Drop" entry.
- **Swap courses:** pick a catalog section and the enrolled section of the same course to replace, run the clash check, then remove the old row and add the new one as two separate, non-atomic steps [**flagged**]: if adding the replacement fails after the old section was already removed, nothing rolls the deletion back.
- **View/export schedule:** Day mode shows only today's sections (weekday name computed from the system clock); Week mode shows everything and enables an Excel export via pandas/openpyxl, which auto-increments the output filename (`schedule_by_week (1).xlsx`, etc.) rather than overwrite an existing file. Day-mode export is deliberately disabled — the README explains it previously produced misaligned columns, so it now shows an "Export Not Available" message instead.
- **Attendance:** a table seeded from **four hardcoded dummy rows**, not a real database table — there is no **Attendance** model. Marking a row Present/Absent and confirming emails the student a status change and logs it, but the underlying table resets to the same dummy data every time the screen reopens [**a stated, in-scope limitation per the README, not something this document infers**].
- **Activity log:** a read-only table of every logged action for the current student, populated from the shared **Activity** table that every other screen writes into as a side effect of its own actions.

5 Key design decisions and their trade-offs

Email is optional, and the app is honest about it. Both the login-verification and attendance-confirmation email paths check for the two required environment variables *before* attempting any SMTP connection, and fall back to a truthful status (printed code; "confirmation emails could not be sent") rather than hanging or silently claiming success. This lets anyone clone and run the project without the author's Gmail App Password.

One data-access class, one big caveat. Every screen goes through **AllQueries** rather than writing SQL of its own, which is a clean separation of UI from data logic. The catch, named directly by the project's own README as a lesson learned, is that **AllQueries** tracks the "current student" as a mutable instance attribute (`glob_id`, defaulting to a hardcoded id) that each screen overwrites on open, rather than passing a student/session object explicitly — correctness depends on call order rather than being structurally guaranteed. The README's own proposed fix (a shared session object owned by the main window) was not implemented.

Plaintext passwords. The `Student.password` column is a plain string and login uses a bare `==` comparison — no hashing. The README states this directly as "a real weakness, not a design choice worth defending" and names bcrypt hashing as follow-up work, not a silent gap.

Relative .ui paths. Every screen loads its layout with a path relative to the working directory, so

the app (and, per the README, its manual smoke-testing process) only works when launched from the project root — the reason the Docker image sets an explicit `WORKDIR` and the run scripts assume the project directory.

6 What's flagged, not fabricated

- The README's testing section describes a headless smoke-test method (`QT_QPA_PLATFORM=offscreen` plus monkey-patched `QMessageBox` dialogs) used to verify every screen builds and queries correctly. **No test file of any kind exists in the repository as reviewed** — no `test_*.py`, no `tests/` directory. Whether this method was run ad hoc and never committed, or removed later, cannot be determined from the source alone.
- `courses_database.py` and `add_remove_interaction.ui` are present in the repo but unused by any other file — confirmed by searching every import and `loadUi` call.
- The `Course.swap_id/remove_id` columns and the entire `Filter` table are declared in the schema but never queried anywhere.
- `swap_courses` performs its remove-then-add as two separate database operations, not one transaction.

7 What the authors say they'd change

Directly from the README's "What I'd do differently": hash passwords with `bcrypt`; replace the shared, mutable `glob_id` with an explicitly-passed session/student object; use one shared `AllQueries` instance and DB session across the app instead of each screen constructing its own; give attendance a real, persisted database model instead of hardcoded dummy rows; replace scattered `print` debugging with logging, and stop `student_database.py` from running `create_all` and SQL-echoing as a side effect of being imported; add automated tests.

8 Glossary

ORM — library mapping database rows to code objects (here, `SQLAlchemy`). **SQLite** — a serverless, single-file relational database. **MFA** — multi-factor authentication; more than one proof of identity to log in. **SMTP** — the protocol used to send email (via Python's `smtpplib` here). **Data-access layer** — one class/module through which all other code reads and writes the database (here, `AllQueries`), keeping SQL out of the UI code. **Qt Designer / .ui file** — an XML layout file for a window's widgets, built visually and loaded at runtime instead of hand-coded.